

Documentación y Manual de Usuario

Introducción

El presente documento describe el funcionamiento del software PathBench, una aplicación desarrollada en Python cuyo propósito es analizar y comparar el rendimiento de algoritmos de búsqueda de rutas más cortas en grafos.

El sistema fue diseñado utilizando:

- Python
- Tkinter
- Matplotlib

La aplicación permite generar grafos dinámicos y ejecutar pruebas comparativas entre los algoritmos:

- Dijkstra
- Bellman-Ford

Además, el sistema mide distintos indicadores de rendimiento como:

- Tiempo de ejecución.
- Consumo de memoria.
- Número de iteraciones.
- Distancia mínima encontrada.

Este proyecto tiene como objetivo demostrar el comportamiento computacional de ambos algoritmos dependiendo del tamaño y complejidad del grafo generado.

Objetivo del Software

El objetivo principal del programa es proporcionar una herramienta visual e interactiva para el análisis de algoritmos de rutas más cortas mediante pruebas de benchmarking.

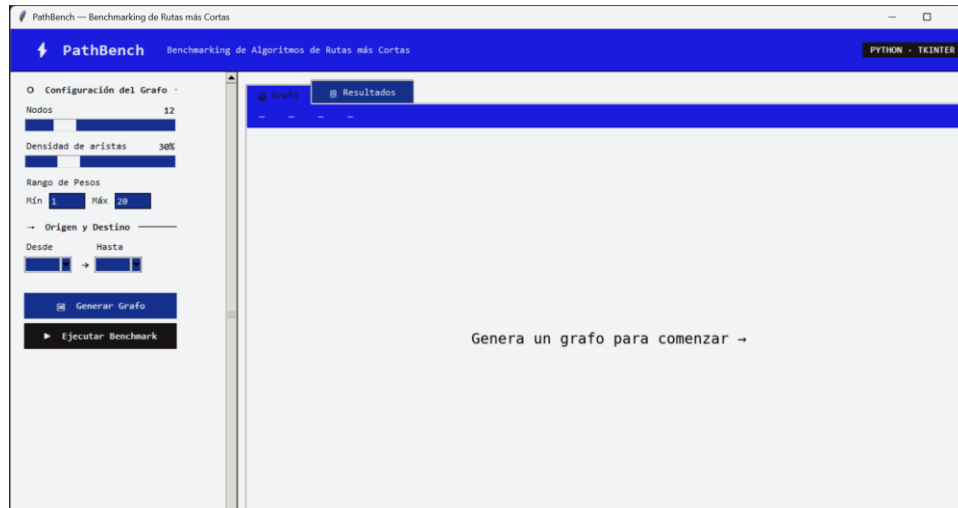
El sistema permite:

- Generar grafos aleatorios.
- Configurar densidad y pesos de las aristas.
- Ejecutar algoritmos de búsqueda.
- Comparar eficiencia computacional.

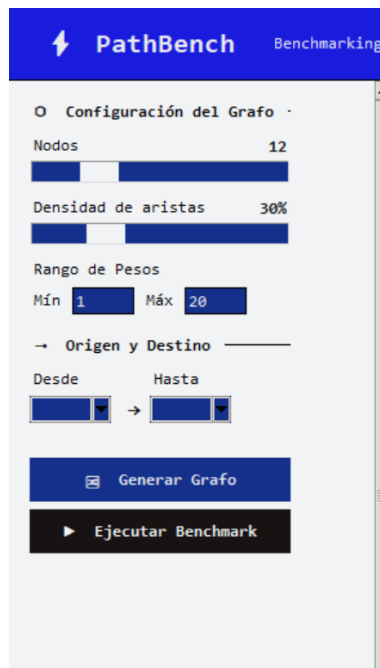
- Visualizar gráficamente las rutas obtenidas.

Ejecución del programa

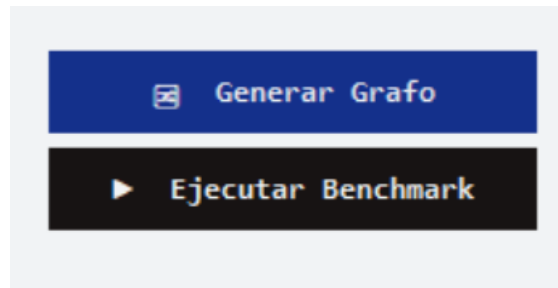
Al momento de ejecutar el programa, el usuario visualizará una interfaz gráfica denominada PathBench, la cual fue diseñada para permitir la interacción de manera sencilla e intuitiva con los algoritmos de rutas más cortas.



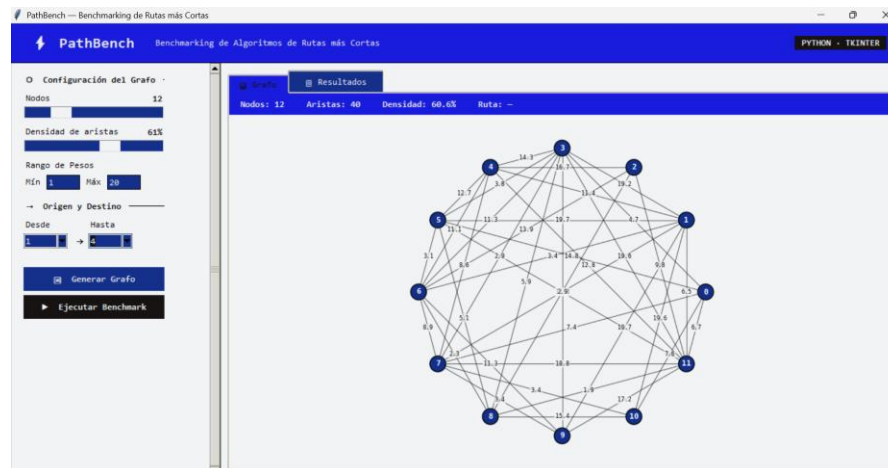
Inicialmente, el usuario deberá configurar los parámetros del grafo desde el panel lateral izquierdo, donde podrá seleccionar la cantidad de nodos, el porcentaje de densidad de aristas y el rango de pesos.



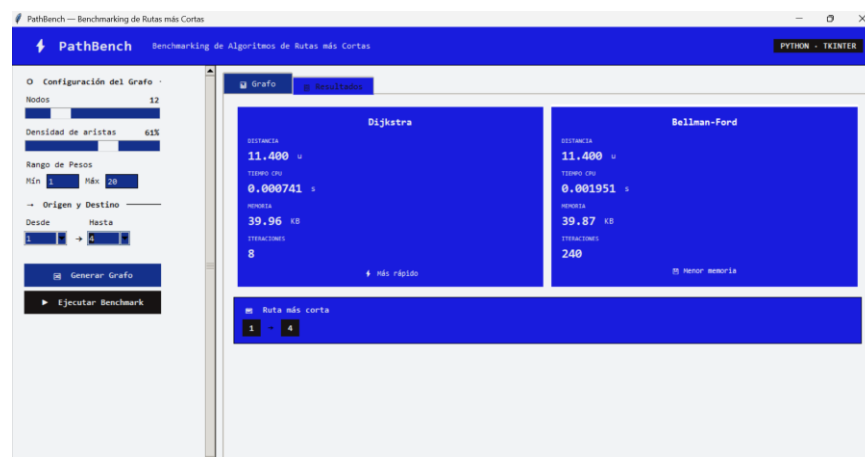
Una vez configurados los parámetros, se deberá presionar el botón “Generar Grafo”, el cual creará automáticamente un grafo aleatorio y lo representará visualmente dentro de la ventana principal del sistema.



Posteriormente, el usuario tendrá que seleccionar el nodo de origen y el nodo destino mediante los cuadros desplegables correspondientes. Estos nodos representan el punto inicial y final sobre los cuales se calculará la ruta más corta.



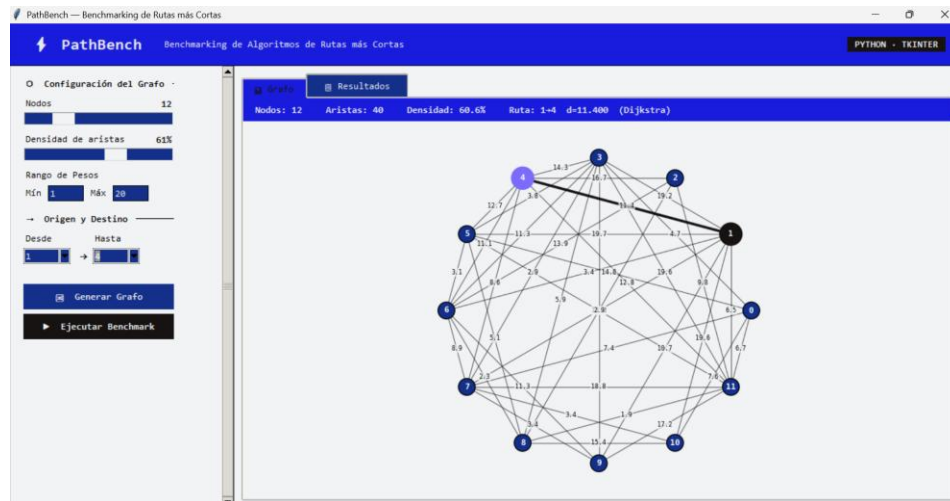
Después de seleccionar ambos nodos, el usuario presionará el botón “Ejecutar Benchmark”, provocando que el sistema ejecute los algoritmos de Dijkstra y Bellman-Ford para realizar el análisis comparativo de rendimiento.



Finalmente, el sistema mostrará una sección de resultados donde se podrán observar diferentes métricas, tales como:

- Distancia mínima encontrada.
- Tiempo de ejecución de cada algoritmo.
- Consumo de memoria.
- Número de iteraciones realizadas.
- Ruta más corta obtenida.

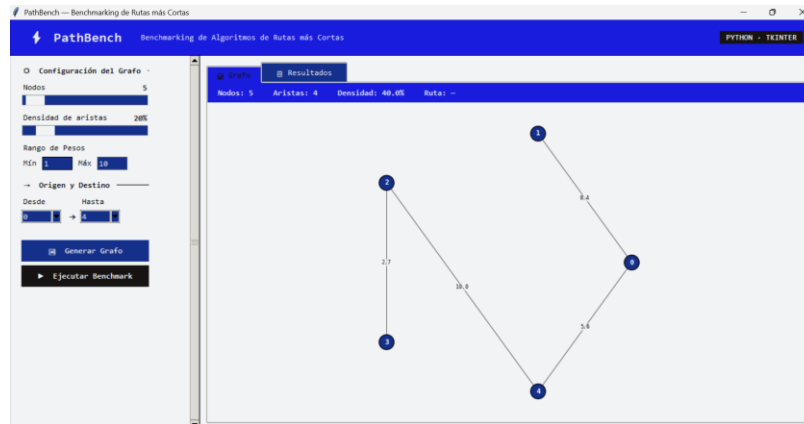
Adicionalmente, la ruta calculada será resaltada gráficamente dentro del grafo para facilitar la interpretación visual de los resultados obtenidos.



EJEMPLOS DE USO

Grafo pequeño

Comenzaremos con un grafo pequeño los valores que podemos poner son las siguientes cantidades: nodos 5, densidad 20%, Rango de pesos 1 a 10.



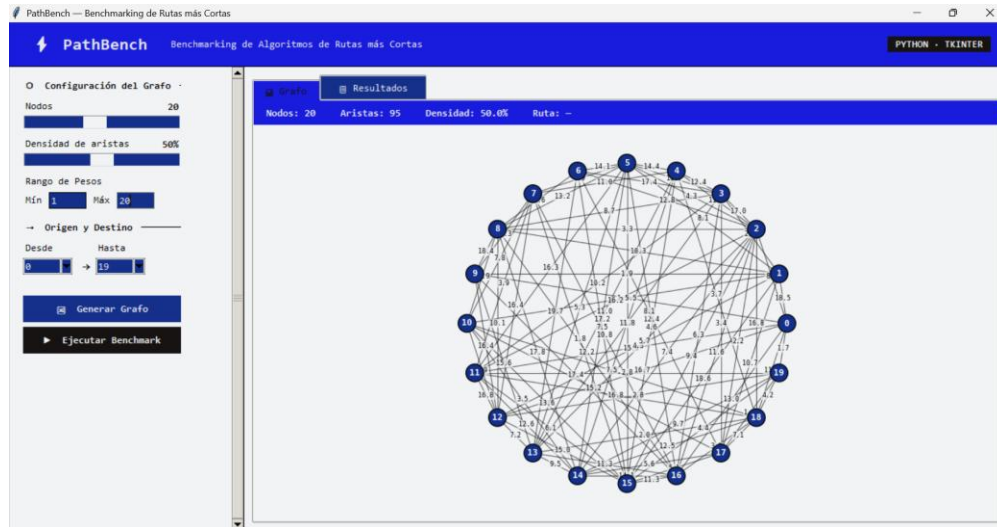
Después de generar el grafo, podemos continuar ejecutando el programa para obtener los resultados del análisis. El sistema mostrará información importante como el tiempo de ejecución, la memoria utilizada, la distancia de la ruta más corta y el número de iteraciones realizadas por cada algoritmo.

En este ejemplo, los resultados fueron rápidos y el consumo de memoria fue bajo debido a que el grafo contaba con pocos nodos y pocas conexiones. Ambos algoritmos lograron encontrar la ruta más corta correctamente; sin embargo, el algoritmo de Dijkstra presentó un mejor rendimiento, ya que es más eficiente en grafos pequeños con pesos positivos.

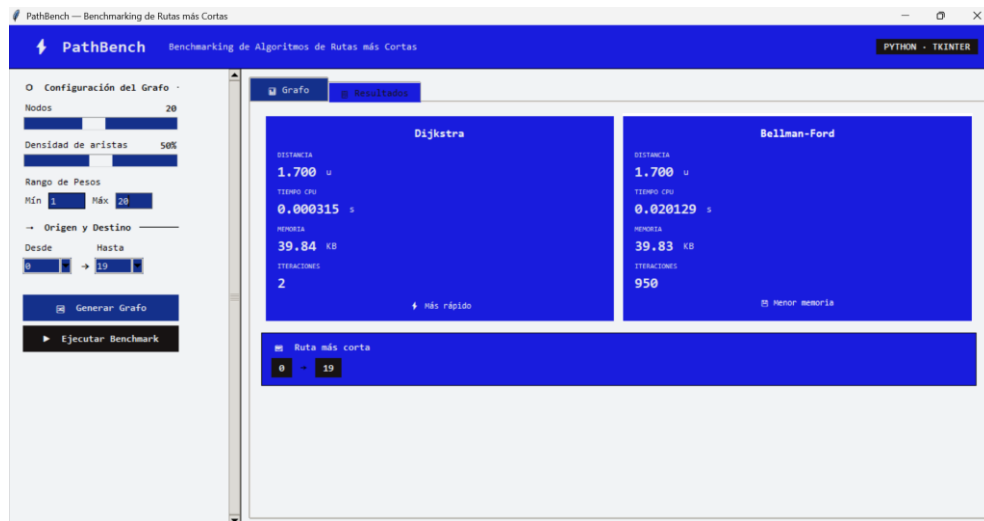
Resultados	
Dijkstra	Bellman-Ford
DISTANCIA 5.600 u	DISTANCIA 5.600 u
TIEMPO CPU 0.000208 s	TIEMPO CPU 0.000533 s
MEMORIA 39.86 KB	MEMORIA 39.85 KB
ITERACIONES 2	ITERACIONES 24
⚡ más rápido	💾 mejor memoria
Ruta más corta 0 → 4	

Grafo mediano

Ahora continuamos con un grafo un poco más complejo para realizar una comparación más detallada entre los algoritmos. Para esta prueba se pueden utilizar los siguientes valores de configuración: 20 nodos, una densidad de aristas del 50 % y un rango de pesos de 1 a 20.

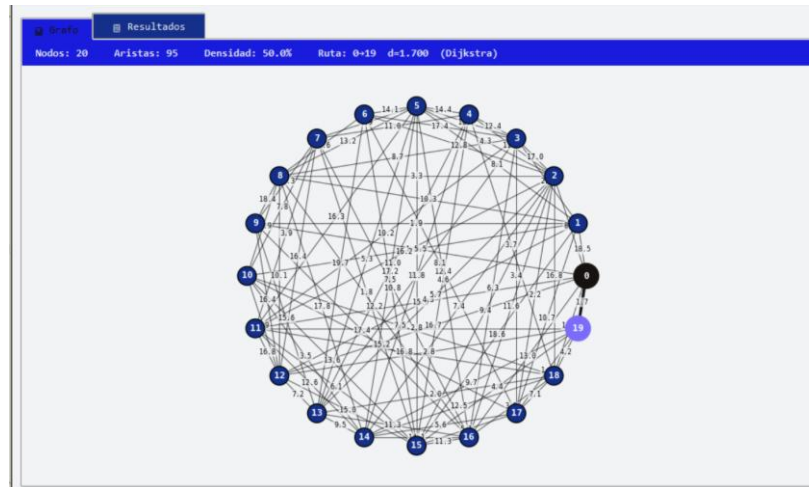


Una vez configurado el grafo podemos ejecutar el programa para que nos muestre los resultados obtenidos por cada algoritmo.



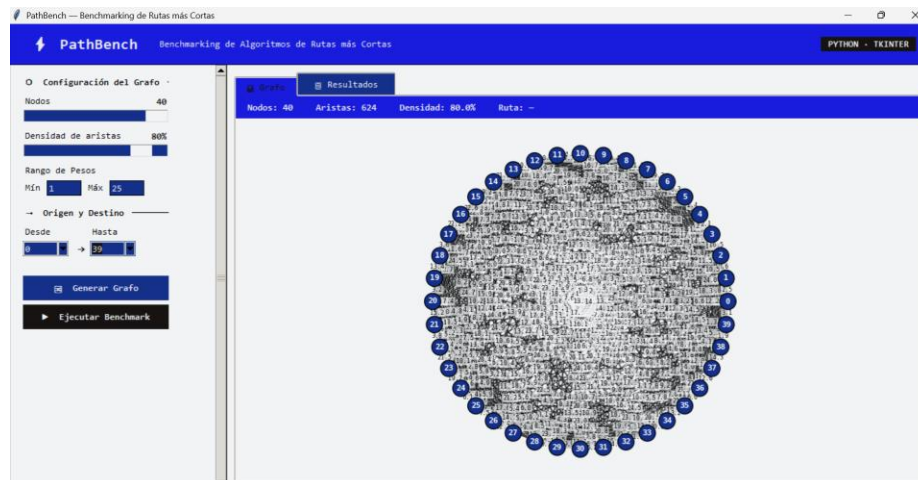
Como se puede observar, el sistema presenta información diferente para cada método, como la distancia calculada, el tiempo de ejecución, la memoria utilizada y el número de iteraciones realizadas durante el proceso. Estos resultados permiten comparar el rendimiento y la eficiencia de cada algoritmo al trabajar con un grafo más complejo.

Además, en la representación gráfica del grafo se muestra visualmente la ruta más corta encontrada, permitiendo identificar de manera más clara el recorrido óptimo entre el nodo de origen y el nodo destino, como se observa en la siguiente imagen.

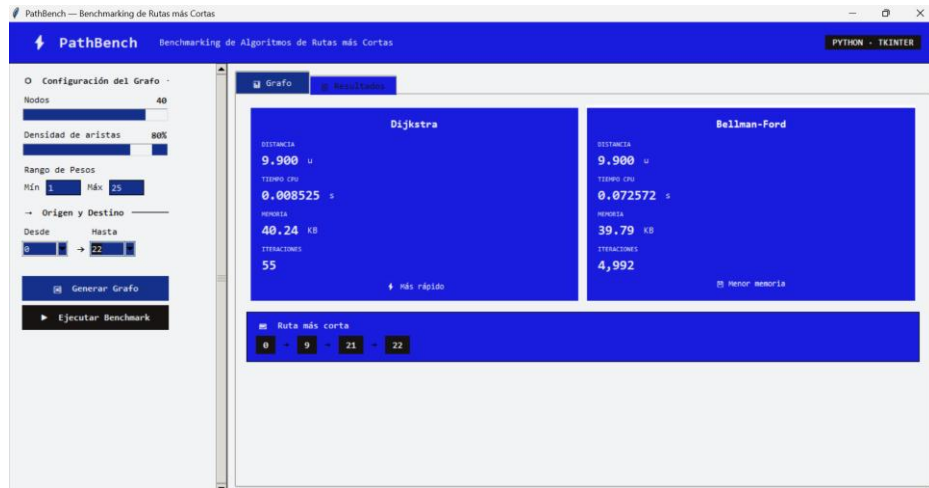


Grafo grande

Ahora continuamos con un grafo de mayor complejidad. Para esta prueba se pueden utilizar los siguientes valores de configuración: 40 nodos, una densidad del 80 % y un rango de pesos de 1 a 25.



Al igual que en los ejemplos anteriores, después de generar el grafo podemos ejecutar el programa para obtener los resultados correspondientes de cada algoritmo.



En este tercer grafo, los resultados mostraron un mayor tiempo de ejecución y un mayor consumo de memoria debido a la complejidad del grafo y a la gran cantidad de conexiones entre nodos.

